Synchronous FIFO

Name: Gautam Raj Chhabra
Project: Synchronous FIFO
Language: Verilog HDL
Tool Used: Vivado Xilinx

1. Abstract

This project implements a Synchronous FIFO (First-In First-Out) memory using Verilog HDL. The FIFO operates on a single clock and supports simultaneous read and write operations. The design uses pointers and control logic to manage data flow efficiently. The FIFO ensures proper handling of full and empty conditions and maintains data integrity. The design is verified using a testbench and simulated waveform. RTL, synthesis, and implementation schematics are generated using Vivado.

2. Introduction

A FIFO (First-In First-Out) is a memory structure widely used in digital systems for buffering data between modules operating at the same clock.
In this project, a synchronous FIFO is designed where both read and write operations occur on the same clock. The design includes:

- Memory array
- Write pointer
- Read pointer
- Control logic for full and empty detection

The FIFO stores data sequentially and retrieves it in the same order. A testbench is used for verification, and simulation ensures correct functionality.

3. Operational Conditions

- WRITE: Data is written into FIFO when write_en = 1 and FIFO is not full
- READ: Data is read from FIFO when read_en = 1 and FIFO is not empty
- FULL: FIFO cannot accept new data
- EMPTY: FIFO has no data to read

4. Design Code

```
module fifo(
```

```
  input clk,
  input reset,
  input write_en,
  input read_en,
  input [7:0] data_in,
  output reg [7:0] data_out,
  output full,
  output empty
);

  parameter DEPTH = 8;
  parameter ADDR_WIDTH = 3;

  reg [7:0] mem [0:DEPTH-1];

  reg [ADDR_WIDTH-1:0] write_ptr;
  reg [ADDR_WIDTH-1:0] read_ptr;

  reg [ADDR_WIDTH:0] count;  // 0 to DEPTH
  assign full  = (count == DEPTH);
  assign empty = (count == 0);

  wire write_valid = write_en && (count < DEPTH);
  wire read_valid  = read_en  && (count > 0);

  always @(posedge clk or posedge reset) begin
    if (reset) begin
      write_ptr <= 0;
    end
    else if (write_valid) begin
      mem[write_ptr] <= data_in;
      write_ptr <= (write_ptr == DEPTH-1) ? 0 : write_ptr
+ 1;
    end
  end


  always @(posedge clk or posedge reset) begin
    if (reset) begin
```

```
      read_ptr <= 0;
      data_out <= 0;
    end
    else if (read_valid) begin
      data_out <= mem[read_ptr];
      read_ptr <= (read_ptr == DEPTH-1) ? 0 : read_ptr +
1;
    end
  end

  always @(posedge clk or posedge reset) begin
    if (reset) begin
      count <= 0;
    end
    else begin
      case ({write_valid, read_valid})
        2'b10: count <= count + 1; // write only
        2'b01: count <= count - 1; // read only
        2'b11: count <= count;     // both
        2'b00: count <= count;
      endcase
    end
  end

endmodule
```

5. Testbench

`timescale 1ns/1ps

```
module testbench;

reg clk;
reg reset;
reg write_en;
reg read_en;
reg [7:0] data_in;

wire [7:0] data_out;
wire full;
```

```
wire empty;


// DUT

fifo uut (
    .clk(clk),
    .reset(reset),
    .write_en(write_en),
    .read_en(read_en),
    .data_in(data_in),
    .data_out(data_out),
    .full(full),
    .empty(empty)
);


// clock generation

always #5 clk = ~clk;


// test

initial
begin

    clk = 0;
    reset = 1;
    write_en = 0;
    read_en = 0;
    data_in = 0;

    #10 reset = 0;


    // WRITE DATA

    #10 write_en = 1; data_in = 8'd10;
    #10 data_in = 8'd20;
```

```
    #10 data_in = 8'd30;
    #10 data_in = 8'd40;
    #10 data_in = 8'd50;
    #10 data_in = 8'd60;
    #10 data_in = 8'd70;
    #10 data_in = 8'd80;

    #10 write_en = 0;


    // READ DATA

    #10 read_en = 1;
    #80 read_en = 0;


    #20 $finish;

end


// monitor

initial
begin

  $dumpfile("dump.vcd");
  $dumpvars(0,testbench);
    $monitor("time=%0t write=%b read=%b data_in=%d data_out=%d full=%b
empty=%b",
            $time, write_en, read_en, data_in, data_out, full, empty);
end


endmodule
```

6. Simulation Result

The FIFO design is simulated to verify correct data storage and retrieval.

- Data written into FIFO is read in the same order
- Full flag is asserted when FIFO is full

- Empty flag is asserted when FIFO is empty

The simulation confirms proper working of read/write operations and pointer logic.

```
Tcl Console | Messages | Log | Reports | Design Runs | DRC | Methodology | Hard Block Planner | Power | Timing

Time resolution is 1 ps
time=0 write=0 read=0 data_in=  0 data_out=  0 full=0 empty=1
time=20000 write=1 read=0 data_in= 10 data_out=  0 full=0 empty=1
time=25000 write=1 read=0 data_in= 10 data_out=  0 full=0 empty=0
time=30000 write=1 read=0 data_in= 20 data_out=  0 full=0 empty=0
time=40000 write=1 read=0 data_in= 30 data_out=  0 full=0 empty=0
time=50000 write=1 read=0 data_in= 40 data_out=  0 full=0 empty=0
time=60000 write=1 read=0 data_in= 50 data_out=  0 full=0 empty=0
time=70000 write=1 read=0 data_in= 60 data_out=  0 full=0 empty=0
time=80000 write=1 read=0 data_in= 70 data_out=  0 full=0 empty=0
time=90000 write=1 read=0 data_in= 80 data_out=  0 full=0 empty=0
time=95000 write=1 read=0 data_in= 80 data_out=  0 full=1 empty=0
time=100000 write=0 read=0 data_in= 80 data_out=  0 full=1 empty=0
time=110000 write=0 read=1 data_in= 80 data_out=  0 full=1 empty=0
time=115000 write=0 read=1 data_in= 80 data_out= 10 full=0 empty=0
time=125000 write=0 read=1 data_in= 80 data_out= 20 full=0 empty=0
time=135000 write=0 read=1 data_in= 80 data_out= 30 full=0 empty=0
time=145000 write=0 read=1 data_in= 80 data_out= 40 full=0 empty=0
time=155000 write=0 read=1 data_in= 80 data_out= 50 full=0 empty=0
time=165000 write=0 read=1 data_in= 80 data_out= 60 full=0 empty=0
time=175000 write=0 read=1 data_in= 80 data_out= 70 full=0 empty=0
time=185000 write=0 read=1 data_in= 80 data_out= 80 full=0 empty=1
time=190000 write=0 read=0 data_in= 80 data_out= 80 full=0 empty=1
$finish called at time : 210 ns : File "C:/vivado/vending/project_6/project_6.srcs/sim_1/new/eda_fifo_tb.v" Line 69
```

7. Waveform

The waveform illustrates the functional behavior of the Synchronous FIFO during simulation.

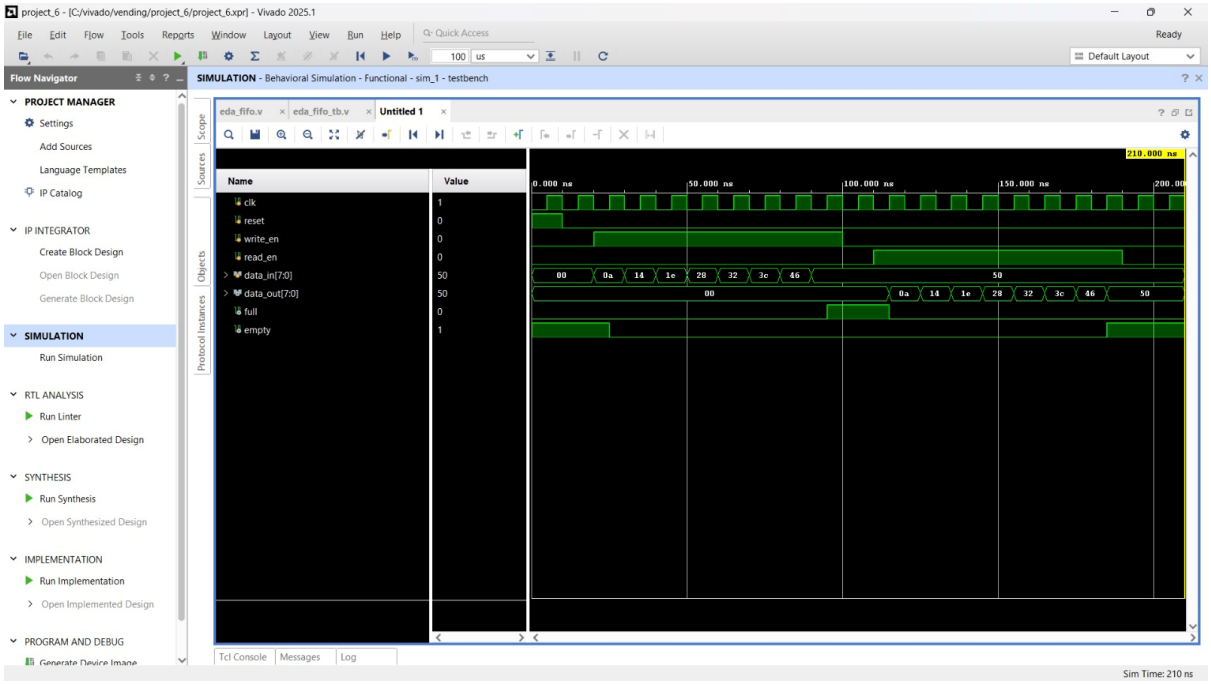
Initially, the system is in reset state, where the FIFO is empty (empty = 1) and not full (full = 0). After the reset is deasserted, write operations begin when write_en is asserted. Data is written sequentially into the FIFO through data_in. As soon as the first data is written, the empty signal goes low, indicating that the FIFO now contains valid data.

As multiple write operations occur, the FIFO continues to store data until it reaches its maximum capacity. At this point, the full signal is asserted, preventing further writes and ensuring no overflow occurs.

During the read phase, read_en is asserted and data is read from the FIFO through data_out. The data appears in the same order as it was written, confirming correct FIFO (First-In First-Out) behavior. As data is read, the full signal is deasserted, and once all stored data has been read, the empty signal is asserted again.

The waveform confirms:

- Correct sequential data write and read operations
- Proper assertion and deassertion of full and empty signals
- Accurate FIFO behavior with maintained data order

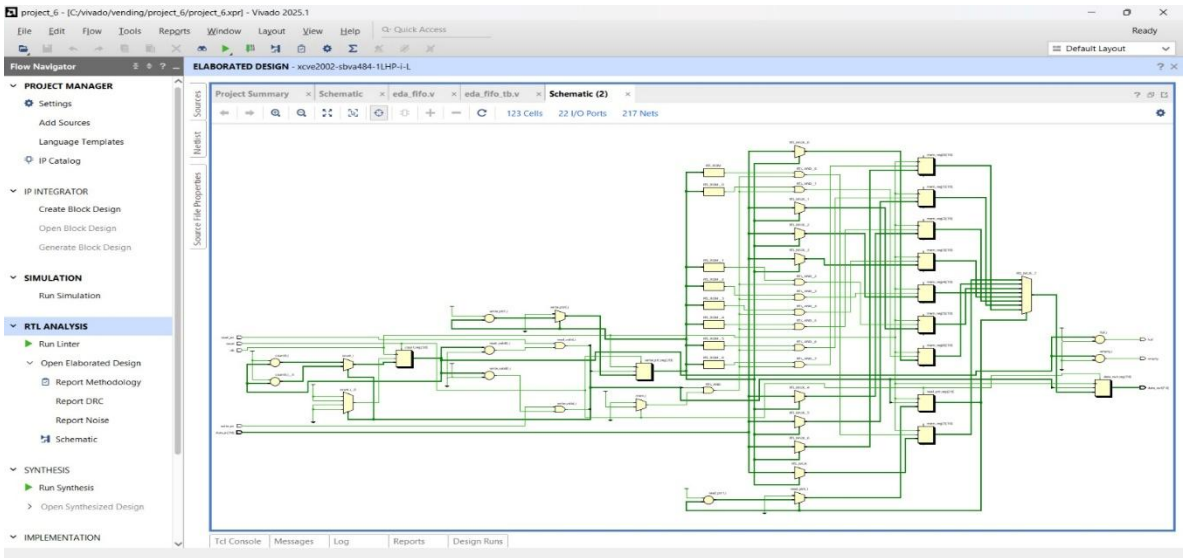


8. RTL Schematic

The RTL schematic represents the structural view of the FIFO design. It includes:

- Memory block
- Read/Write pointers
- Control logic

This confirms correct module integration



9. Conclusion

The Synchronous FIFO is successfully implemented using Verilog HDL.

- Correct data ordering is maintained
- Full and empty conditions are handled properly
- Simulation and schematics validate the design